

React Interview Prep

Topic 15 — Error Boundaries & Portals: Complete Guide

Fuse Box Analogy · Error Boundary Class · `getDerivedStateFromError` · `componentDidCatch` · Placement · Limits · Portal · `createPortal` · Event Bubbling · Q&A;

- Covers All examples from conversation — nothing skipped
- Series React Interview Prep — Session 15

by Akshay Chavhan • React Interview Series

1. Like You're 5 — Two Analogies

■ Error Boundaries — Fuse Box

Your house has fuse boxes in each room.

If one room shorts — only that fuse trips.

Rest of the house stays lit ■

Without boundaries → 1 crash = whole app dark

With boundaries → 1 crash = only that section dark

■ Portals — Poster on Outside Wall

You're inside a room and need to put a

poster on the OUTSIDE wall of the building

— not inside your room.

Portal = render output somewhere ELSE in DOM

but still controlled by React ■

PART 1 — Error Boundaries

2. The Problem Without Error Boundaries

■ Without Error Boundary

```
function BrokenComponent() {  
  throw new Error('crashed!'); // ■  
  return <div>Hello</div>;  
}  
  
function App() {  
  return (  
    <div>  
      <Navbar /> // works  
      <BrokenComponent /> // ■ crashes  
      <Footer /> // never renders  
    </div>  
  );  
}  
  
// Entire screen = white blank page  
■
```

■ With Error Boundary

```
// ■ With Error Boundary  
function App() {  
  return (  
    <div>  
      <Navbar />  
      <ErrorBoundary>  
        <BrokenComponent /> // crashes  
      </ErrorBoundary>  
      <Footer /> // still works ■  
    </div>  
  );  
}  
  
// Only the wrapped section shows  
// error fallback — rest is fine ■
```

3. Error Boundary Class Component

Error Boundaries must be class components — `getDerivedStateFromError` and `componentDidCatch` are class lifecycle methods. React has not released a hook-based equivalent yet.

```
import React from 'react';  
  
class ErrorBoundary extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { hasError: false, error: null };  
  }  
  
  // Runs when child throws — update state to show fallback  
  static getDerivedStateFromError(error) {
```

```
return { hasError: true, error };
}

// Runs after error – good for logging to Sentry etc
componentDidCatch(error, errorInfo) {
  console.error('Error caught:', error);
  console.error('Stack:', errorInfo.componentStack);
  // logToSentry(error, errorInfo); // send to error tracking
}

render() {
  if (this.state.hasError) {
    return (
      <div className='error-fallback'>
        <h2>Something went wrong</h2>
        <p>{this.state.error?.message}</p>
        <button onClick={() => this.setState({ hasError: false })}>
          Try Again
        </button>
      </div>
    );
  }
  return this.props.children; // render children normally ■
}
}
```

4. Where to Place Error Boundaries

```
// Strategy 1 – Top level (catches everything as last resort)
<ErrorBoundary>
<App />
</ErrorBoundary>

// Strategy 2 – Route level (each page isolated)
<Route path='/dashboard' element={
  <ErrorBoundary><Dashboard /></ErrorBoundary>
} />

// Strategy 3 – Widget level (granular – best UX) ■
<ErrorBoundary fallback={<p>Widget failed</p>}>
<LiveStockTicker />
</ErrorBoundary>

// Real apps use all 3 layers together!
// Top = last resort | Route = page isolation | Widget = granular
```

Strategy	What it protects	User experience
Top level	Entire app	Fallback for catastrophic errors
Route level	Each page/route	Other pages still work if one crashes
Widget level	Individual sections	Best UX — rest of page still works

5. What Error Boundaries CANNOT Catch

Cannot catch	Reason	Fix instead
Event handler errors	Not during render — React can't intercept	Use try/catch inside handler
Async errors (setTimeout/fetch)	Thrown after render cycle is done	Use try/catch inside async functions
Errors in boundary itself	Can't catch its own errors	Need parent Error Boundary
Server-side rendering errors	Error Boundaries are client-only	Handle in SSR error handling
■ CAN catch: render errors	Errors thrown during component render	Wrap with ErrorBoundary

■ **Error Boundaries ONLY catch errors during render, lifecycle methods, and constructors. Event handlers and async code need their own try/catch.**

6. react-error-boundary — Production Pattern

Instead of writing the class component manually, use the popular react-error-boundary library.

```
import { ErrorBoundary } from 'react-error-boundary';
function ErrorFallback({ error, resetErrorBoundary }) {
  return (
```

```

<div>
  <p>Something went wrong: {error.message}</p>
  <button onClick={resetErrorBoundary}>Try Again</button>
</div>
);
}

function App() {
  return (
    <ErrorBoundary
      fallbackComponent={ErrorFallback}
      onError={(error, info) => logToSentry(error, info)}
      onReset={() => { /* reset any related state */ }}
    >
      <Dashboard />
    </ErrorBoundary>
  );
}

```

■ *react-error-boundary* library is the production standard. Cleaner API, `resetKeys` prop, `onReset` callback — no manual class component needed.

7. The Problem Without Portals

■ Without

```
// ■ Modal trapped inside
overflow:hidden

function Card() {
  return (
    <div style={{
      overflow: 'hidden',
      position: 'relative'
    }}>
      {/* modal gets CLIPPED here ■ */}
      <Modal />
    </div>
  );
}
```

■ With

```
// ■ Portal escapes to
document.body

function Card() {
  return (
    <div style={{
      overflow: 'hidden',
      position: 'relative'
    }}>
      {/* modal renders in body ■ */}
      <Modal />
    </div>
  );
}

// No clipping – full screen modal
■
```

8. createPortal — Syntax & Basic Example

```
import { createPortal } from 'react-dom';

function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;
  // Renders directly inside <body> – outside Card's overflow! ■
  return createPortal(
    <div className='modal-overlay' onClick={onClose}>
      <div className='modal-content'
        onClick={e => e.stopPropagation()}>
        {children}
      </div>
    </div>,
    document.body // ← target DOM node to render into
  );
}

function Card() {
  const [isOpen, setIsOpen] = useState(false);
  return (
    <div style={{ overflow: 'hidden' }}>
      <button onClick={() => setIsOpen(true)}>Open Modal</button>
      <Modal isOpen={isOpen} onClose={() => setIsOpen(false)}>
        <h2>I am outside the Card!</h2> {/* renders in body ■ */}
      </Modal>
    </div>
  );
}
```


9. DOM Structure — Before vs After Portal

■ Without Portal — modal trapped

```
<body>
<div id='root'>
  <div class='card'
    style='overflow:hidden'>
    <button>Open</button>
    <div class='modal-overlay'> ←
    TRAPPED, gets clipped ■
    <div
      class='modal-content'>Hello</div>
    </div>
  </div>
</div>
</body>
```

■ With Portal — modal at body level

```
<body>
<div id='root'>
  <div class='card'
    style='overflow:hidden'>
    <button>Open</button>
    ← modal NOT here anymore
  </div>
</div>
<div class='modal-overlay'> ← in
body ■
<div
  class='modal-content'>Hello</div>
</div>
</body>
```

10. Event Bubbling Still Works Through Portals

Even though portal renders outside the parent in the DOM, React events still bubble through the component tree — not the DOM tree.

```
function Parent() {
  return (
    <div onClick={() => console.log('Parent clicked!')}>
      <Modal />
    </div>
    /* Modal renders in document.body — outside Parent's div in DOM */
    /* BUT clicking inside Modal STILL bubbles to Parent ■ */
    /* because React uses VIRTUAL component tree, not real DOM tree */
  );
}
```

■ **Portal children are still logical children of the parent in React's component tree. Events bubble through React's virtual tree regardless of DOM placement.**

11. Other Portal Use Cases

Use Case	Why Portal?	Example
Modal / Dialog	Escapes overflow:hidden parent	Confirmation dialog, image lightbox
Tooltip	Escapes overflow + z-index issues	Hover tooltip on table cell
Toast Notification	Always on top, fixed position	Success/error toast corner
Dropdown Menu	Escapes table or card overflow	Select dropdown, context menu

12. Production Modal Pattern — Full Implementation

```
function Modal({ isOpen, onClose, title, children }) {  
  // Lock scroll when modal is open  
  useEffect(() => {  
    if (isOpen) document.body.style.overflow = 'hidden';  
    return () => { document.body.style.overflow = 'unset'; };  
  }, [isOpen]);  
  
  // Close on Escape key  
  useEffect(() => {  
    function handleKey(e) { if (e.key === 'Escape') onClose(); }  
    document.addEventListener('keydown', handleKey);  
    return () => document.removeEventListener('keydown', handleKey);  
  }, [onClose]);  
  
  if (!isOpen) return null;  
  
  return createPortal(  
    <div  
      role='dialog' aria-modal='true' aria-label={title}  
      style={{ position:'fixed', inset:0, background:'rgba(0,0,0,0.5)',  
        display:'flex', alignItems:'center', justifyContent:'center',  
        zIndex: 1000 }}  
      onClick={onClose}  
    >  
      <div style={{ background:'white', borderRadius:8, padding:24 }}  
        onClick={e => e.stopPropagation()}>  
        <h2>{title}</h2>  
        {children}  
        <button onClick={onClose}>Close</button>  
      </div>  
    </div>,  
    document.body  
  );  
}
```

13. Interview Q&A;

Q

Beginner

Q1. What is an Error Boundary?

→ An Error Boundary is a React class component that catches JavaScript errors in its child component tree, logs them, and displays a fallback UI instead of crashing the entire app. It uses `getDerivedStateFromError` to show the fallback and `componentDidCatch` to log the error.

Q

Beginner

Q2. Why must Error Boundaries be class components?

→ Because `getDerivedStateFromError` and `componentDidCatch` are class lifecycle methods. React hasn't released a hook-based equivalent yet. In practice most teams use the `react-error-boundary` library which wraps this class component with a much cleaner functional API.

Q

Intermediate

Q3. What can't Error Boundaries catch?

→ They cannot catch errors in event handlers — use `try/catch` there. They cannot catch async errors like promises or `setTimeout`. They cannot catch errors in the boundary itself — need a parent boundary. They only catch errors during render, lifecycle methods, and constructors of child components.

Q

Intermediate

Q4. What is a Portal?

→ A Portal renders a component's output into a different DOM node than its parent using `createPortal(children, domNode)`. The component is still part of React's component tree logically so events bubble normally, but in the real DOM it renders somewhere else entirely — like directly in `document.body`.

Q

Intermediate

Q5. When would you use a Portal?

→ Modals, tooltips, dropdowns, and toasts — any UI that needs to visually break out of its parent's CSS constraints like `overflow:hidden` or `z-index` stacking contexts. Without a portal, these get clipped or hidden. The portal renders directly in `document.body` bypassing those constraints.

Q

Expert

Q6. Does event bubbling work through Portals?

→ Yes — even though a portal renders outside its parent in the DOM, React events still bubble through the component tree not the DOM tree. Clicking inside a modal portal will bubble up to its React parent. This is intentional — the portal child is still a logical child of the parent in React's virtual component tree.

Q

Expert

Q7. How do you handle multiple Error Boundary layers in production?

→ Use three layers: top-level boundary as last resort to prevent complete white screen, route-level boundaries to isolate each page, and widget-level boundaries for risky third-party or data-dependent components. Use react-error-boundary library with `onError` prop to log to Sentry or similar tracking.